

Yesterday 3:41 PM

How does HashMap work? — excluded from this list because you asked to skip Collections.
How does equals()/hashCode() work?
synchronized vs Lock vs volatile.
How do you identify and prevent deadlocks?
How does ExecutorService work?
How does CompletableFuture work?
thenApply() vs thenCompose().
Sequential vs parallel streams.
map() vs flatMap().
orElse() vs orElseGet().

Show more ▾



Absolutely. For a **5-year Java interview**, these are exactly the kinds of questions where the interviewer expects more than a definition. You should explain **concept** → **internal behavior** → **real-world usage** → **trade-offs** → **example**.

Below is an interview-ready version.

1. How does equals() / hashCode() work?

Interview answer

equals() determines whether two objects are logically equal, while hashCode() returns an integer used by hash-based data structures.

The important contract is:

If two objects are equal according to equals() , they **must have the same hashCode**.

But the reverse is not required:

Two objects can have the same hashCode but still not be equal.

Example

</> Java



```
class Employee {  
    private int id;  
    private String name;  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
  
        if (!(obj instanceof Employee)) return false;  
  
        Employee other = (Employee) obj;  
  
        return id == other.id &&  
            Objects.equals(name, other.name);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(id, name);  
    }  
}
```

Why both are needed?

Suppose a hash-based structure wants to find an object.

Conceptually:

```
object  
  ↓  
hashCode()  
  ↓  
bucket  
  ↓  
equals()  
  ↓  
matching object
```



`hashCode()` helps locate the candidate area, and `equals()` determines whether the candidate is actually equal.

Common interview trap

If you override `equals()` but not `hashCode()` :

</> Java

```
Employee e1 = new Employee(1, "Manoj");
Employee e2 = new Employee(1, "Manoj");

e1.equals(e2); // true
```

But they may have different hash codes inherited from `Object` , violating the contract.

5-year point

Fields used in `equals()` / `hashCode()` should ideally be **stable/immutable** while the object is being used as a key in a hash-based structure.

2. synchronized VS Lock VS volatile

This is a very common 5-year interview question.

Feature	synchronized	Lock	volatile
Mutual exclusion	Yes	Yes	No
Visibility	Yes	Yes	Yes
Atomic compound operation	Yes	Yes	No
Explicit lock/unlock	No	Yes	No
tryLock()	No	Yes	No
Interruptible locking	Limited	Yes	No
Typical use	Simple synchronization	Advanced concurrency	Visibility

synchronized

</> Java 

```
public synchronized void increment() {  
    count++;  
}
```

Only one thread can execute the synchronized section for the same monitor at a time.

It provides:

- mutual exclusion
- visibility
- happens-before guarantees

Lock

</> Java 

```
private final Lock lock = new ReentrantLock();  
  
public void increment() {  
    lock.lock();  
  
    try {  
        count++;  
    } finally {  
        lock.unlock();  
    }  
}
```

Advantages:

</> Java 

```
if (lock.tryLock(1, TimeUnit.SECONDS)) {  
    try {  
        // work  
    } finally {  
        lock.unlock();  
    }  
}
```

Useful when you need:

- tryLock()
 - timeout
 - interruptible locking
 - multiple conditions
 - more explicit control
-

volatile

</> Java



```
private volatile boolean running = true;
```

If one thread changes:

</> Java



```
running = false;
```

other threads can see the updated value.

But:

</> Java



```
volatile int count;  
  
count++;
```

is **not atomic**.

Because:

```
read count  
↓  
add 1  
↓  
write count
```



Multiple threads can interleave these operations.

Good use case

</> Java



```
private volatile boolean shutdown;
```

For a simple shared state flag, `volatile` can be appropriate.

Strong interview answer

`volatile` provides visibility and ordering guarantees but not mutual exclusion or general compound-operation atomicity. `synchronized` and `Lock` provide mutual exclusion and visibility.

3. How do you identify and prevent deadlocks?

A deadlock occurs when threads wait indefinitely for resources held by each other.

Example:

Thread 1:

Lock A → waiting for Lock B



Thread 2:

Lock B → waiting for Lock A

Neither can continue.

Code

</> Java



```
synchronized (lockA) {  
  
    synchronized (lockB) {  
        // work  
    }  
}
```

Another thread:

</> Java



```
synchronized (lockB) {  
  
    synchronized (lockA) {  
        // work  
    }  
}
```

Potential deadlock.

How do you identify it?

In production, take a **thread dump**.

For example:

```
</> Bash  
  
jstack <pid>
```



The dump can show threads waiting for monitors/locks and may identify deadlocked threads.

You can also use tools such as:

- JConsole
- VisualVM
- Java Flight Recorder
- Mission Control
- application/APM monitoring

How do you prevent it?

1. Consistent lock ordering

Always acquire:

```
Lock A → Lock B
```



Never:

Lock B → Lock A



2. Use tryLock()

</> Java



```
if (lock.tryLock(2, TimeUnit.SECONDS)) {  
    try {  
        // work  
    } finally {  
        lock.unlock();  
    }  
}
```

3. Keep critical sections small

Don't perform unnecessary:

DB call
HTTP call
file operation



while holding a lock.

4. Avoid unnecessary nested locks

5-year interview statement

I would first confirm the issue using thread dumps and lock information. Then I would identify the lock cycle, fix lock ordering or reduce nested locking, and potentially use timed tryLock() where appropriate.

4. How does ExecutorService work?

ExecutorService provides a framework for executing tasks using managed threads rather than manually creating threads.

Instead of:

</> Java



```
new Thread(() -> doSomething()).start();
```

we can use:

</> Java



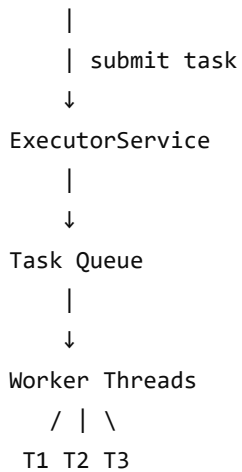
```
ExecutorService executor =
    Executors.newFixedThreadPool(10);

executor.submit(() -> doSomething());

executor.shutdown();
```

Conceptually:

Application



submit()

Returns a Future .

</> Java



```
Future<Integer> future =
    executor.submit(() -> calculate());

Integer result = future.get();
```

Important concepts

A thread pool manages:

- worker threads
- task queue

- task execution
- lifecycle
- rejection behavior

Why thread pools?

Creating a new thread for every request is expensive and can cause resource exhaustion.

A pool allows controlled concurrency.

5-year point

Don't blindly say:

Java



```
Executors.newFixedThreadPool(...)
```

In production, thread-pool sizing should consider:

- CPU count
- CPU-bound vs I/O-bound workload
- queue size
- request latency
- downstream capacity
- rejection policy

For more control, configure `ThreadPoolExecutor` explicitly.

5. How does `CompletableFuture` work?

`CompletableFuture` is used for asynchronous and composable operations.

Example:

Java



```
CompletableFuture<String> future =  
    CompletableFuture.supplyAsync(() -> getUser());
```

You can then process the result:

</> Java



```
future.thenApply(user -> user.toUpperCase());
```

Conceptually:

Async operation



CompletableFuture



thenApply



thenApply



thenAccept



Example

</> Java



```
CompletableFuture<User> userFuture =  
    CompletableFuture.supplyAsync(() -> userService.getUser());  
  
CompletableFuture<String> result =  
    userFuture  
        .thenApply(User::getName)  
        .thenApply(String::toUpperCase);
```

The important benefit is that operations can be **composed without blocking the calling thread**.

Combining independent calls

Suppose:

User API
Order API
Payment API



are independent.

We can execute:

`</>` Java



```
CompletableFuture<User> user =  
    CompletableFuture.supplyAsync(() -> getUser());  
  
CompletableFuture<List<Order>> orders =  
    CompletableFuture.supplyAsync(() -> getOrders());  
  
CompletableFuture<Payment> payment =  
    CompletableFuture.supplyAsync(() -> getPayment());
```

Then:

`</>` Java



```
CompletableFuture.allOf(user, orders, payment);
```

This allows the independent operations to run concurrently.

6. thenApply() vs thenCompose()

This is a **very important** interview question.

thenApply()

Use when the next operation returns a normal value.

`</>` Java



```
CompletableFuture<User> userFuture = getUser();  
  
CompletableFuture<String> nameFuture =  
    userFuture.thenApply(User::getName);
```

Conceptually:

```
Future<User>  
  ↓  
thenApply  
  ↓  
Future<String>
```



thenCompose()

Use when the next operation itself returns a `CompletableFuture` .

Suppose:

</> **Java**

```
CompletableFuture<User> getUser()
```



and:

</> **Java**

```
CompletableFuture<List<Order>> getOrders(User user)
```



Then:

</> **Java**

```
CompletableFuture<List<Order>> result =  
    getUser()  
    .thenCompose(user -> getOrders(user));
```



Without `thenCompose()` :

</> **Java**

```
thenApply(user -> getOrders(user))
```



would conceptually produce:

```
CompletableFuture<CompletableFuture<List<Order>>>
```



`thenCompose()` flattens it:

```
Future<User>
```

↓

```
Future<List<Order>>
```



Easy memory trick

thenApply = transform

thenCompose = chain another async operation

7. Sequential vs Parallel Streams

Sequential

</> Java

```
list.stream()
    .map(this::process)
    .toList();
```



Operations normally execute on the calling thread.

Conceptually:

Thread

↓

1 → 2 → 3 → 4 → 5



Parallel

</> Java

```
list.parallelStream()
    .map(this::process)
    .toList();
```



Work can be split among multiple threads, commonly using the common `ForkJoinPool`.

Conceptually:

```

      Data
    /  |  \
   T1 T2 T3
    \  |  /
     Result
```



When parallel streams can help

Good candidate:

- large dataset
- CPU-intensive processing
- independent operations
- enough CPU resources

When they can hurt

Avoid blindly using them for:

- small collections
- blocking I/O
- operations with shared mutable state
- order-sensitive processing
- applications where common-pool contention matters

5-year answer

Parallel streams are not automatically faster. I would benchmark them based on data size, CPU workload, operation cost, and thread-pool contention.

8. map() vs flatMap()

map()

Transforms one element into another.

</> Java

```
List<String> names = List.of("A", "B", "C");

List<Integer> lengths =
    names.stream()
        .map(String::length)
        .toList();
```

A → 1

B → 1

C → 1

flatMap()

Used when each element produces multiple elements or another stream.

Example:

```
</> Java
List<List<String>> data = List.of(
    List.of("A", "B"),
    List.of("C", "D")
);
```

With `map()` :

```
</> Java
data.stream()
    .map(List::stream)
```

gives:

```
Stream<Stream<String>>
```

With `flatMap()` :

```
</> Java
data.stream()
    .flatMap(List::stream)
    .toList();
```

gives:

```
[A, B, C, D]
```

Easy rule

`map()` = one-to-one transformation

`flatMap()` = flatten nested structures

9. `orElse()` vs `orElseGet()`

This is a classic Java 8 interview trap.

</> Java



```
Optional<String> name = Optional.of("Manoj");
```

`orElse()`

</> Java



```
name.orElse(getDefaultName());
```

The default expression is evaluated **even when the Optional contains a value**.

`orElseGet()`

</> Java



```
name.orElseGet(() -> getDefaultName());
```

The supplier is evaluated only when the Optional is empty.

Example

</> Java



```
String result = Optional.of("Java")  
    .orElse(expensiveOperation());
```

`expensiveOperation()` still executes.

Whereas:

</> Java



```
String result = Optional.of("Java")  
    .orElseGet(() -> expensiveOperation());
```

doesn't invoke the expensive operation.

Interview answer

`orElse()` eagerly evaluates its argument, whereas `orElseGet()` lazily evaluates the supplier.

10. How do you create an immutable class?

An immutable object cannot change its state after construction.

Example:

</> Java



```
public final class Employee {  
  
    private final int id;  
    private final String name;  
  
    public Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Important rules:

1. Make class `final`

</> Java



```
final class Employee
```

Prevents subclassing that could introduce mutability.

2. Make fields private and final

</> Java



```
private final String name;
```

3. Initialize through constructor

4. Don't provide setters

5. Defensive copy mutable fields

This is where 5-year candidates should go deeper.

Suppose:

</> Java



```
private final List<String> roles;
```

Don't expose the original list.

</> Java



```
public Employee(List<String> roles) {  
    this.roles = List.copyOf(roles);  
}  
  
public List<String> getRoles() {  
    return roles;  
}
```

For mutable objects, use appropriate defensive copying.

Why immutable objects are useful?

They are easier to:

- share between threads
- reason about
- cache
- use safely as keys
- test

11. How do you apply SOLID principles in a real project?

Don't just give definitions.

Use an actual example.

Suppose you're designing an order/payment system.

S — Single Responsibility

Bad:

```
</> Java
```

```
OrderService
```



doing:

```
validate order  
calculate price  
save DB  
send email  
generate invoice
```



Better:

```
OrderValidator  
PricingService  
OrderRepository  
NotificationService  
InvoiceService
```



Each has one primary responsibility.

O — Open/Closed

Suppose payment supports:

```
CreditCard  
UPI  
PayPal
```



Don't keep modifying a giant `if/else`.

Use:

</> Java



```
interface PaymentProcessor {  
    void pay(Payment payment);  
}
```

Implement:

</> Java



```
class CardPaymentProcessor implements PaymentProcessor  
class UpiPaymentProcessor implements PaymentProcessor  
class PaypalPaymentProcessor implements PaymentProcessor
```

Adding a new payment method doesn't require changing existing implementations extensively.

L — Liskov Substitution

If:

</> Java



```
Bird
```

has:

</> Java



```
fly()
```

then making:

</> Java



```
Penguin extends Bird
```

can violate LSP because penguins don't fly.

The abstraction is wrong.

Design interfaces based on actual capabilities.

I — Interface Segregation

Instead of:

</> Java



```
interface Employee {  
    work();  
    eat();  
    manage();  
    generateReport();  
}
```

split interfaces where appropriate:

</> Java



```
interface Worker {  
    void work();  
}  
  
interface Manager {  
    void manage();  
}
```

Clients shouldn't depend on methods they don't need.

D — Dependency Inversion

Instead of:

</> Java



```
class OrderService {  
    private MySQLOrderRepository repository =  
        new MySQLOrderRepository();  
}
```

depend on an abstraction:

</> Java



```
class OrderService {  
  
    private final OrderRepository repository;  
  
    OrderService(OrderRepository repository) {  
        this.repository = repository;  
    }  
}
```

Now Spring can inject:

```
MySQLOrderRepository  
PostgresOrderRepository  
MockOrderRepository
```



Strong 5-year answer

In real projects, SOLID helps control coupling and makes components easier to test, replace and extend. I don't apply the principles mechanically; I use them where they improve maintainability and reduce change impact.

12. How would you design a thread-safe Singleton?

One strong implementation is the Initialization-on-demand holder idiom:

</> Java



```
public final class Singleton {  
  
    private Singleton() {  
    }  
  
    private static class Holder {  
        private static final Singleton INSTANCE =  
            new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return Holder.INSTANCE;  
    }  
}
```

Why is this good?

- lazy initialization
- thread-safe class initialization
- no explicit synchronization
- simple implementation

Another good approach: enum

</> Java



```
public enum Singleton {  
    INSTANCE;  
}
```

This provides strong guarantees around instance creation and serialization.

Interview warning

This is unsafe:

</> Java



```
if (instance == null) {  
    instance = new Singleton();  
}
```

Two threads can simultaneously see `null` .

Double-checked locking can work with `volatile` :

</> Java



```
private static volatile Singleton instance;  
  
public static Singleton getInstance() {  
  
    if (instance == null) {  
        synchronized (Singleton.class) {  
  
            if (instance == null) {  
                instance = new Singleton();  
            }  
        }  
    }  
}
```

```
}  
  
    return instance;  
}
```

But for most situations, the holder idiom or enum is simpler.

13. How would you handle failures in concurrent API calls?

This is a very good 5-year microservices question.

Suppose you call:

```
User Service  
Order Service  
Payment Service  
Recommendation Service
```



concurrently.

Don't allow one failure to unexpectedly bring down everything.

With `CompletableFuture` :

```
</> Java  
  
CompletableFuture<User> user =  
    getUser();  
  
CompletableFuture<List<Order>> orders =  
    getOrders();  
  
CompletableFuture<List<Product>> recommendations =  
    getRecommendations();
```



Then handle failure:

```
</> Java  
  
recommendations.exceptionally(ex -> {  
    log.error("Recommendation service failed", ex);  
    return Collections.emptyList();  
});
```



Or:

</> Java



```
future.handle((result, exception) -> {
    if (exception != null) {
        // fallback
    }

    return result;
});
```

Production design

I would consider:

Timeout



Retry (only where safe)



Circuit Breaker



Fallback



Logging/Metrics/Tracing



For example:

Recommendation API



timeout



retry?



circuit breaker



fallback = []



Important

Don't blindly retry everything.

For a non-idempotent operation like:

Create payment



a retry could potentially create duplicate business effects unless idempotency is designed properly.

5-year answer

For concurrent API calls, I separate independent calls, execute them concurrently, apply timeouts, handle individual failures independently, and provide fallbacks where the business allows it. For transient failures I use controlled retries and circuit breakers, while ensuring idempotency for retryable operations.

14. How would you investigate a slow/hanging Java API in production?

This is probably the most important scenario question in your list.

Don't immediately say:

"I will check the logs."

Give a structured investigation.

Step 1 — Confirm the symptom

Check:

Latency
Error rate
Throughput
CPU
Memory
GC
Thread count
DB latency
External API latency



Use monitoring/APM.

First determine:

Is everything slow?
or
Only one endpoint?



Step 2 — Check application metrics

Suppose:

CPU = 30%
Memory = normal
GC = normal



but:

API latency = 5 seconds



That immediately tells me CPU saturation may not be the primary issue.

Possible causes:

DB waiting
HTTP downstream waiting
thread pool exhaustion
connection pool exhaustion
lock contention
queue backlog
network latency



Step 3 — Take thread dumps

For a hanging Java application:

</> Bash

jstack <pid>



Look for states such as:

BLOCKED
WAITING
TIMED_WAITING



Questions I'd ask:

Are many threads blocked?
Are they waiting for the same lock?
Are they waiting for DB connections?
Are they waiting on HTTP calls?
Is there a deadlock?



Step 4 — Check thread-pool exhaustion

Suppose:

Tomcat request threads = 200



and all 200 are waiting for a downstream API.

New requests cannot be processed efficiently.

This can produce:

CPU low
Memory normal
Response time high



This is a classic production issue.

Step 5 — Check DB connection pool

Suppose HikariCP has:

maximumPoolSize = 20



and all 20 connections are busy.

Requests wait for connections.

Application CPU can remain normal while latency increases.

So check:

```
active connections
idle connections
pending threads
connection acquisition time
slow queries
```



Step 6 — Check downstream APIs

For every external dependency:

```
Service A
Service B
Database
Redis
Kafka
```



check:

```
latency
timeout
error rate
connection pool
```



Distributed tracing is extremely useful here.

For example:

```
Client
  ↓ 20ms
API
  ↓ 30ms
Service A
  ↓ 3.5 sec
Payment Service
```



Now you know where the latency is.

Step 7 — Check GC

Look for:

frequent GC
long GC pauses
heap pressure
allocation spikes



But don't assume:

"Java API is slow → GC problem."

You need evidence.

Step 8 — Check locks/deadlocks

Use:

thread dump
JFR
JConsole
VisualVM



Look for:

BLOCKED threads
lock contention
deadlock



Step 9 — Compare before vs after deployment

If the problem started immediately after deployment:

What changed?



Check:

new code
new SQL



```
configuration
thread pool size
connection pool
timeouts
external API
JVM version
GC configuration
feature flags
```

A deployment correlation is a major clue, but not proof.

Step 10 — Mitigate first, investigate deeply second

In production, don't spend 2 hours investigating while users are suffering.

Depending on the situation:

```
Rollback
Scale instances
Disable feature flag
Increase downstream capacity
Fix configuration
Reduce traffic
```



Then perform the deeper root-cause analysis.

A very strong interview answer

If the interviewer asks:

"Your Java API suddenly becomes slow in production. How will you investigate?"

You can answer:

"First, I would quantify the issue using latency, throughput and error-rate metrics and determine whether it's endpoint-specific or system-wide. Then I'd correlate it with recent deployments or configuration changes. I would check CPU, memory and GC, but also look at thread-pool utilization, database connection-pool exhaustion, downstream service latency and lock contention. If requests are hanging, I'd take thread dumps and check for BLOCKED, WAITING and deadlocked threads. I'd use distributed tracing or APM to identify which dependency is consuming the latency. Once I identify the bottleneck, I'd first apply a production mitigation such as rollback, scaling or disabling

a feature if necessary, and then fix the root cause and add monitoring to prevent recurrence."

That answer sounds much more like a **5-year production engineer** than simply saying "I'll check logs."

★ What you should prioritize for a 5-year interview

From this entire list, I'd rank them:

● Must master

1. `equals()` / `hashCode()`
2. `synchronized` VS `Lock` VS `volatile`
3. Deadlock identification/prevention
4. `ExecutorService` / `ThreadPoolExecutor`
5. `CompletableFuture`
6. `thenApply()` VS `thenCompose()`
7. Concurrent API failure handling
8. Production slow API investigation

● Very important

9. Sequential vs parallel streams
10. `map()` VS `flatMap()`
11. `orElse()` VS `orElseGet()`
12. Immutable class
13. Thread-safe Singleton

● Design/interview discussion

14. SOLID principles in a real project

For **5 years of Java/Spring Boot experience**, don't prepare these as isolated definitions. Prepare each one in this format:

Definition → **Internal working** → **Code** → **Real production example** → **Problem/trade-off** → **What I would do in production.**

That is the level that will differentiate a 5-year candidate from a 2–3-year candidate.



I use CompletableFuture for parallel calls.

Why? → Which executor? → Which thread executes it? →
What if one fails? → Timeout? → 1,000 requests/sec? →
How do you prevent thread-pool exhaustion?

What is CompletableFuture

Why do we need CompletableFuture when ExecutorService already exists?

Show more ∨

Yes — this is exactly the **follow-up chain an interviewer can use for CompletableFuture**. For a 5-year candidate, don't stop at "it runs calls in parallel." You should be ready to explain **executor choice, thread execution, failure, timeout, throughput, and pool exhaustion**.

1. What is CompletableFuture ?

CompletableFuture is a Java API for representing the result of an **asynchronous computation** and composing multiple asynchronous operations.

It implements Future and CompletionStage .

Simple example:

</> Java



```
CompletableFuture<String> future =  
    CompletableFuture.supplyAsync(() -> callUserService());  
  
future.thenApply(user -> user.toUpperCase())  
    .thenAccept(System.out::println);
```

Think of it as:

Start async operation



```

CompletableFuture
    ↓
transform result
    ↓
another async operation
    ↓
handle result/error

```

The important feature is **composition**.

You can build an asynchronous pipeline such as:

```

User API  ————┐
                  └─> combine —> response
Order API  ————┐
                  |
Payment API ————┘

```

2. Why do we need `CompletableFuture` when `ExecutorService` already exists?

This is an excellent interview question.

`ExecutorService`

Primarily answers:

"Where/how should my task execute?"

</> **Java**

```

ExecutorService executor =
    Executors.newFixedThreadPool(10);

Future<User> future =
    executor.submit(() -> getUser());

```

You get a `Future` , but composing multiple asynchronous operations is more cumbersome.

For example:

</> Java



```
Future<User> user = executor.submit(...);
Future<Order> order = executor.submit(...);
```

You generally end up doing blocking calls such as:

</> Java



```
user.get();
order.get();
```

CompletableFuture

Answers:

"How should asynchronous operations be composed?"

For example:

</> Java



```
CompletableFuture<User> user =
    CompletableFuture.supplyAsync(() -> getUser(), executor);

CompletableFuture<List<Order>> orders =
    CompletableFuture.supplyAsync(() -> getOrders(), executor);

CompletableFuture<String> result =
    user.thenCombine(orders,
        (u, o) -> u.getName() + " " + o.size());
```

You can easily express:

A → B → C



or:

```

A ┌──┐
B ┌──┴───> D
C ┌──┐
  
```



and failure handling:

</> Java



```
future.exceptionally(ex -> fallback());
```

Interview answer

`ExecutorService` is primarily a task execution and thread-pool abstraction, whereas `CompletableFuture` provides asynchronous result handling and composition. They are complementary rather than alternatives. In production, I often use `CompletableFuture` with a properly configured `ExecutorService`.

3. When would you use `CompletableFuture`?

The strongest use case is independent I/O calls that can execute concurrently.

Suppose your API needs:

User Service	300 ms
Order Service	400 ms
Recommendation	500 ms



Sequential execution:

```
300 + 400 + 500  
= 1200 ms
```



If independent and executed concurrently:

```
max(300, 400, 500)  
≈ 500 ms
```



Conceptually:

</> Java



```
CompletableFuture<User> user =  
    CompletableFuture.supplyAsync(  
        () -> userService.getUser(), executor);  
  
CompletableFuture<List<Order>> orders =  
    CompletableFuture.supplyAsync(  
        () -> orderService.getOrders(), executor);
```

```

        () -> orderService.getOrders(), executor);

    CompletableFuture<List<Product>> products =
        CompletableFuture.supplyAsync(
            () -> recommendationService.getProducts(), executor);

```

Then combine them.

Good use cases

- parallel REST calls
- parallel database calls where appropriate
- independent service calls
- asynchronous processing
- composing dependent async calls
- applying timeouts/fallbacks

Don't use it just because it is "modern"

If the operation is simple and sequential:

</> **Java**



```

User user = getUser();
return user.getName();

```

there is usually no reason to introduce `CompletableFuture` .

4. Why `thenCompose()` instead of `thenApply()` ?

This is one of the most important questions.

Suppose:

</> **Java**



```

CompletableFuture<User> getUser()

```

and:

</> **Java**



```

CompletableFuture<List<Order>> getOrders(User user)

```

You want:

```
getUser()
↓
getOrders(user)
```



Use:

</> Java



```
CompletableFuture<List<Order>> result =
    getUser()
        .thenCompose(user -> getOrders(user));
```

Why?

Because `getOrders()` already returns a `CompletableFuture`.

With `thenApply()`:

</> Java



```
getUser()
    .thenApply(user -> getOrders(user));
```

you conceptually get:

```
CompletableFuture<
    CompletableFuture<List<Order>>
>
```



That's nested.

`thenCompose()` flattens it:

```
Future<User>
↓
Future<List<Order>>
```



Easy interview trick

`thenApply()` → transform the result.

`thenCompose()` → call another asynchronous operation and flatten the resulting future.

5. Which thread executes the callback?

This is where many candidates give an incomplete answer.

There are three important methods:

`</>` Java



```
thenApply()  
thenApplyAsync()  
thenApplyAsync(..., executor)
```

`thenApply()`

The callback generally executes in the thread that completes the previous stage, if the previous stage is already completed, it may execute in the thread calling the continuation.

Example:

`</>` Java



```
future.thenApply(result -> process(result));
```

Don't assume:

"It always runs on the same thread."

The execution depends on completion timing and how the stage is triggered.

`thenApplyAsync()`

`</>` Java



```
future.thenApplyAsync(result -> process(result));
```

The callback is scheduled asynchronously, normally using the `ForkJoinPool.commonPool()` if you don't provide an executor.

Better production approach

Provide your own executor:

</> Java



```
future.thenApplyAsync(  
    result -> process(result),  
    myExecutor  
);
```

Now you've explicitly controlled the executor used for that asynchronous continuation.

Strong interview answer

`thenApply()` is a non-async continuation and may execute in the thread that completes the previous stage. `thenApplyAsync()` schedules the continuation asynchronously, using the common pool by default or the executor I explicitly provide. In production I prefer an appropriately sized application executor when I need isolation and control.

6. What happens when one asynchronous task fails?

Suppose:

</> Java



```
CompletableFuture<User> user = getUser();  
  
CompletableFuture<List<Order>> orders = getOrders();  
  
CompletableFuture<List<Product>> products =  
    getRecommendations();
```

Suppose:

User	SUCCESS
Orders	SUCCESS
Recommendations	FAILURE



The failed future completes **exceptionally**.

You can handle it:

</> Java



```
products.exceptionally(ex -> {
    log.error("Recommendation failed", ex);
    return Collections.emptyList();
});
```

Or:

</> Java



```
products.handle((result, exception) -> {
    if (exception != null) {
        return Collections.emptyList();
    }

    return result;
});
```

7. What happens with `allOf()` ?

This is an important follow-up.

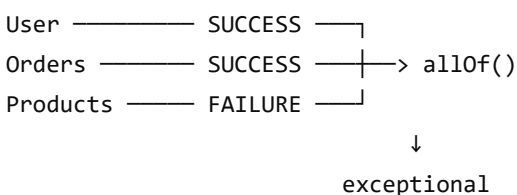
</> Java



```
CompletableFuture<Void> all =
    CompletableFuture.allOf(user, orders, products);
```

If one future completes exceptionally, the combined future also completes exceptionally.

Conceptually:



But don't confuse this with cancellation of all the other tasks.

One failure does not automatically mean all other underlying tasks are cancelled.

If your business requirement is:

"If payment fails, I don't need recommendations."

then you need to design that cancellation/short-circuit behavior explicitly.

8. How do you handle failures in parallel API calls?

For independent APIs, I usually decide which dependencies are:

Mandatory

Example:

User
Payment



If they fail → request should fail.

Optional

Example:

Recommendations
Analytics



If they fail → return a fallback.

For example:

</> Java



```
CompletableFuture<List<Product>> recommendations =  
    getRecommendations()  
        .exceptionally(ex -> {  
            log.error("Recommendation service failed", ex);  
            return Collections.emptyList();  
        });
```

So:

Mandatory failure



Fail request

Optional failure



Fallback



Return partial response

That's much stronger than simply saying:

"I catch the exception."

9. What about timeout?

A very important production concern.

Modern Java provides:

</> Java



```
future.orTimeout(2, TimeUnit.SECONDS);
```

This completes the future exceptionally if it doesn't complete within the timeout.

Or:

</> Java



```
future.completeOnTimeout(  
    fallbackValue,  
    2,  
    TimeUnit.SECONDS  
);
```

That gives you a fallback value after the timeout.

For example:

</> Java

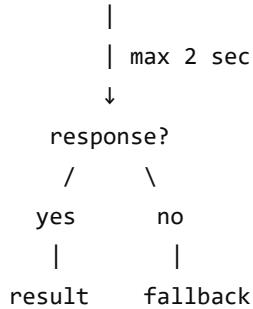


```
CompletableFuture<List<Product>> recommendations =  
    getRecommendations()  
        .completeOnTimeout(  
            Collections.emptyList(),
```

```
2,  
    TimeUnit.SECONDS  
);
```

Conceptually:

Recommendation API



10. What if I have 1,000 requests/sec?

This is where a 5-year interviewer will push you.

Suppose each incoming request makes:

3 parallel downstream calls

At:

1,000 requests/sec

you could generate roughly:

3,000 downstream operations/sec

Now the question becomes:

Can your executor and downstream systems handle that concurrency?

You cannot simply say:

</> Java

```
Executors.newFixedThreadPool(1000);
```

That can make things much worse.

11. How do you prevent thread-pool exhaustion?

This is the key production answer.

1. Use a bounded executor

Don't allow unlimited task accumulation.

For example:

 **Java** 

```
ThreadPoolExecutor executor =  
    new ThreadPoolExecutor(  
        20,  
        50,  
        60,  
        TimeUnit.SECONDS,  
        new ArrayBlockingQueue<>(500),  
        new ThreadPoolExecutor.CallerRunsPolicy()  
    );
```

The exact numbers should come from measurement and workload characteristics, not a universal formula.

2. Separate executors by workload

This is very important.

Don't put everything into one pool:

HTTP calls
DB calls
CPU processing
file operations 

Instead:

HTTP Executor
↓ 

```
external API calls
```

```
DB Executor
```



```
database operations
```

```
CPU Executor
```



```
CPU-intensive processing
```

Why?

Imagine your DB calls suddenly block for 10 seconds.

If DB work shares the same executor with everything else:

```
DB calls
```

```
DB calls
```

```
DB calls
```

```
DB calls
```

```
...
```



the executor can become saturated.

Then unrelated work waits too.

This is called **resource isolation**.

12. Don't accidentally use the common pool for blocking I/O

This is another strong follow-up.

If you write:

```
</> Java
```

```
CompletableFuture.supplyAsync(  
    () -> callSlowExternalAPI()  
);
```



without an executor, the async task normally uses:

```
ForkJoinPool.commonPool()
```



For blocking I/O, blindly putting large amounts of work there can create contention with other application work using the common pool.

Better:

```
</> Java
```



```
CompletableFuture.supplyAsync(  
    () -> callSlowExternalAPI(),  
    ioExecutor  
);
```

where `ioExecutor` is designed for that workload.

13. What happens if 1,000 requests/sec all call a slow API?

Suppose:

```
Incoming traffic = 1000 req/sec
```



```
Downstream API latency = 5 sec
```

If your requests synchronously occupy threads waiting for the downstream service, concurrency can grow dramatically.

A useful mental model is:

```
Concurrency ≈ throughput × latency
```



So:

```
1000 requests/sec × 5 sec  
≈ 5000 concurrent in-flight operations
```



This is why simply saying:

"I'll increase the thread pool."

is dangerous.

You need to control:

- concurrency
- queue size
- timeout
- downstream capacity
- connection pools
- rate limiting
- circuit breakers
- backpressure

14. How I would design it in production

Suppose:

GET /dashboard

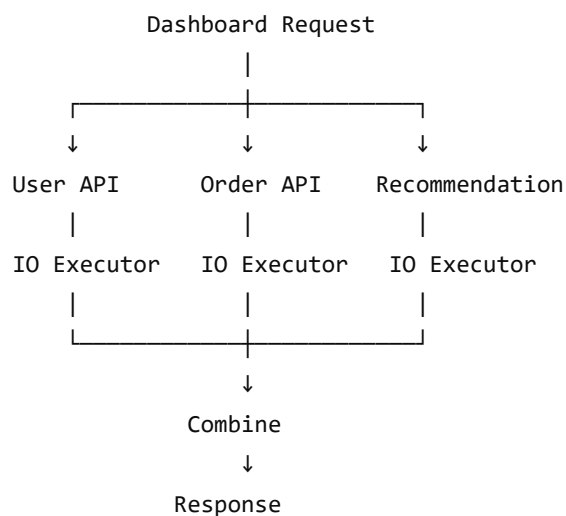


requires:

User Service
Order Service
Recommendation Service



I'd design something conceptually like:



And around those calls:

Timeout



Controlled retry where appropriate



Circuit breaker



Fallback for optional dependencies



Metrics + tracing



Plus:

Bounded executor

Connection-pool limits

Concurrency limits



15. The complete 5-year interview answer

If the interviewer says:

"You said you use `CompletableFuture` for parallel calls. Explain."

A strong answer would be:

"I use `CompletableFuture` when I have independent I/O-bound operations that don't need to execute sequentially. For example, if a dashboard needs user, order and recommendation data, I can initiate those calls concurrently and combine their results rather than waiting for each one sequentially.

I normally provide a dedicated executor rather than blindly using the common `ForkJoinPool`, especially for blocking I/O. The executor is bounded and sized according to the workload, and I monitor active threads, queue depth and task latency.

For dependent asynchronous operations I use `thenCompose()`, while `thenApply()` is for transforming an already available result. For non-async callbacks, `thenApply()` may execute in the thread that completes the previous stage, while `thenApplyAsync()` schedules the callback asynchronously using the common pool or a supplied executor.

For failures, I distinguish between mandatory and optional dependencies. Mandatory failures may fail the request, while optional services can have fallbacks. I also apply timeouts and controlled retries where appropriate. At high traffic, I prevent thread-pool exhaustion with bounded executors, workload isolation, timeouts, connection-pool

limits and concurrency control. I would also monitor downstream latency because increasing the thread pool alone can actually make an overloaded system worse."

The mental model to remember

CompletableFuture



```
|
+-- What?      async result/composition
|
+-- Why?       compose concurrent operations
|
+-- Executor?  control where tasks execute
|
+-- Thread?    depends on async/non-async stage
|
+-- Failure?   exceptionally/handle/whenComplete
|
+-- Timeout?   orTimeout/completeOnTimeout
|
+-- 1000 RPS?  concurrency becomes the problem
|
+-- Exhaustion? bounded pools + isolation
|               + timeouts + backpressure
|
+-- Dependent? thenCompose()
|
+-- Transform? thenApply()
```

This is the level I'd recommend preparing for a 5-year Java/Spring Boot interview: don't memorize `CompletableFuture` methods individually; understand the **threading model + executor + failure + timeout + high-concurrency behavior**.



ChatGPT can make mistakes. Check important info.